

ZHCET Management System

Comprehensive Project Documentation

College Faculty and Student Database Management Application

Owned by: <https://github.com/Mehwash1505>

Repository: https://github.com/AzhaanGlitch/ZHCET_Management

Table of Contents

- 1. Project Overview
- 2. Technology Stack
- 3. Project Directory Structure
- 4. Root Level Files
- 5. Backend Directory - Detailed File Analysis
 - 5.1 server.py - API Server
 - 5.2 database.py - Database Abstraction Layer
 - 5.3 schema.sql - Database Schema and Seed Data
 - 5.4 requirements.txt - Python Dependencies
 - 5.5 .env - Environment Variables
 - 5.6 README.md - Backend Documentation
- 6. Frontend Directory - Detailed File Analysis
 - 6.1 index.html - Landing Page and Authentication
 - 6.2 dashboard.html - Post-Login Dashboard
 - 6.3 style.css - Complete Stylesheet
 - 6.4 app.js - Application Logic
 - 6.5 logo.png - College Logo Asset
 - 6.6 banner.jpg - Banner Image Asset
 - 6.7 .gitignore - Frontend Git Exclusions
- 7. Supabase Integration and Usage
- 8. Authentication Flow
- 9. Role-Based Access Control
- 10. Local Network Access (LAN Server)
- 11. How to Run the Project

1. Project Overview

The ZHCET Management System is a professional, role-based College Faculty and Student Database Management Application built for Zakir Husain College of Engineering and Technology (ZHCET), Aligarh Muslim University. The system provides distinct portals for three user roles: Administrator, Faculty, and Student.

The application follows a strict 'no-framework' development philosophy. The entire frontend is built using pure HTML5, CSS3, and Vanilla JavaScript without any libraries, frameworks, or build tools such as React, Vue, Angular, Tailwind, or Bootstrap. Similarly, the backend is powered by Python's built-in http.server module, avoiding frameworks like Django or Flask.

Key functionalities include:

- Role-Based Access Control (RBAC) with distinct dashboards for Admin, Faculty, and Student.
- Full CRUD operations: Create, Read, Update, Delete records from the admin panel.
- Self-service Sign Up for Faculty and Students.
- Faculty can view their own profile and edit contact details.
- Students can view their personal profile information.
- Search and filter functionality across Faculty and Student directories.
- Professional, institutional UI design with a government-college aesthetic.
- LAN server support allowing access from mobile devices on the same network.

2. Technology Stack

Frontend

- HTML5 - Semantic markup for page structure and content.
- CSS3 - Pure vanilla CSS with CSS custom properties (variables) for theming.
- Vanilla JavaScript (ES6+) - All client-side logic, DOM manipulation, and API calls.
- No frameworks, no libraries, no build tools.

Backend

- Python 3.x - The core runtime language.
- http.server (BaseHTTPRequestHandler) - Python's built-in HTTP server module used as the API server.
- json module - For parsing and serializing JSON request/response bodies.
- urllib.parse - For URL path parsing.

Database

- Supabase (PostgreSQL) - Cloud-hosted PostgreSQL database providing real-time data storage.
- supabase-py - Official Python client library for interacting with the Supabase REST API.
- python-dotenv - For loading environment variables from the .env file.

Deployment and Tools

- Vercel - Frontend static deployment (optional).
- Localtunnel / Ngrok - For generating public URLs to access the local server.
- Git and GitHub - Version control and remote repository hosting.

3. Project Directory Structure

```
ZHCET_Management/
|
|-- .gitignore           # Root-level git exclusion rules
|-- README.md           # Project documentation
|
|-- backend/
|   |-- .env             # Environment variables (Supabase credentials)
|   |-- README.md        # Backend-specific documentation
|   |-- database.py      # Database abstraction layer
|   |-- requirements.txt  # Python package dependencies
|   |-- schema.sql       # SQL schema + seed data
|   |-- server.py        # API HTTP server
|
|-- frontend/
|   |-- .gitignore       # Frontend git exclusion rules
|   |-- index.html       # Landing page and authentication
|   |-- dashboard.html   # Post-login dashboard page
|   |-- style.css        # Complete CSS stylesheet
|   |-- app.js           # All JavaScript application logic
|   |-- logo.png         # College logo image
|   |-- banner.jpg       # Homepage banner image strip
|
|-- venv/                # Python virtual environment (git-ignored)
```

4. Root Level Files

4.1 .gitignore

Located at the project root, this file instructs Git to exclude certain files and directories from version control. It prevents sensitive or auto-generated content from being pushed to the remote GitHub repository.

Excluded patterns:

- venv/ - The Python virtual environment directory containing installed packages.
- .env - Environment variable files containing secret Supabase credentials.
- __pycache__/ - Python's compiled bytecode cache directory.
- *.pyc - Individual compiled Python bytecode files.
- .DS_Store - macOS filesystem metadata files.

4.2 README.md

The primary project documentation file displayed on the GitHub repository page. It provides a professional overview of the project including: key features, the technology stack used, step-by-step installation and setup instructions (database configuration, backend initialization, frontend initialization), and ownership attribution. The README follows a clean, professional format without emojis, adhering to institutional documentation standards.

5. Backend Directory - Detailed File Analysis

5.1 server.py - API Server (112 lines)

This is the core backend file that implements a complete RESTful API server using Python's built-in http.server module. It does NOT use any web framework like Django or Flask.

Architecture:

- Class: RequestHandler extends BaseHTTPRequestHandler to handle all HTTP methods.
- CORS Support: The `_set_headers()` method adds `Access-Control-Allow-Origin: *` headers to every response, enabling the frontend to communicate with the backend across ports.

Supported HTTP Methods and Endpoints:

- OPTIONS (any path): Handles CORS preflight requests from browsers.
- POST `/api/login`: Authenticates users. Accepts JSON body with role, identifier, and password fields. Returns sanitized user data (password_hash removed) on success, or a 401 status on failure.
- POST `/api/{table}`: Creates a new record in the specified table (faculty or student). Used for Sign Up and Admin record creation. Returns 201 on success.
- GET `/api/all/{table}`: Retrieves all records from the specified table. Password hashes are stripped from the response for security.
- PUT `/api/update/{table}/{id_col}/{id_val}`: Updates a specific record identified by its ID column and value. Used for editing faculty/student records.
- DELETE `/api/delete/{table}/{id_col}/{id_val}`: Deletes a specific record from the database.

The server runs on port 8000 by default. The `run()` function binds to all network interfaces (empty string for host), making it accessible on the local network.

5.2 database.py - Database Abstraction Layer (104 lines)

This file provides a clean abstraction layer between the API server and the actual data storage. It implements a dual-mode architecture:

- Primary Mode (Supabase): When valid SUPABASE_URL and SUPABASE_KEY environment variables are present and the supabase Python package is installed, all operations go directly to the cloud Supabase PostgreSQL database.
- Fallback Mode (Mock DB): When Supabase credentials are missing, the system falls back to an in-memory Python dictionary (mock_db) that simulates the database locally for testing purposes.

Functions provided:

- login(role, username_or_email, password): Authenticates a user by role. For admin, matches against username; for faculty/student, matches against email_id.
- get_all(table_name): Returns all records from the specified table.
- update_record(table_name, id_col, id_val, data): Updates a single record matching the given ID column and value with the provided data dictionary.
- create_record(table_name, data): Inserts a new record. In mock mode, auto-generates a UUID for the primary key.
- delete_record(table_name, id_col, id_val): Removes a single record by its identifier.

Current Mock Data (used when Supabase is unavailable):

- Admin: username 'admin', password '000'
- Faculty: Dr. Mohd. Aslam (AI), Dr. Nazia Khatoon (VLSI), Dr. Tameem Ahmad (OOPS)
- Students: Mehwash Nasir (B.Tech CS, Sem 5), Dipanshi Gaur (B.Tech EE, Sem 3)

5.3 schema.sql - Database Schema and Seed Data (49 lines)

This SQL file contains the complete database schema definition and initial seed data. It is designed to be executed directly in the Supabase SQL Editor to set up the cloud database.

Tables defined:

- admin: Fields - admin_id (UUID, PK, auto-generated), username (unique), password_hash.
- faculty: Fields - faculty_id (UUID, PK), name, department, designation, mobile_number, email_id (unique), area_of_specialization, password_hash.
- student: Fields - student_id (UUID, PK), name, course_enrollment, current_semester (INT), email_id (unique), mobile_number, password_hash.

The file also includes INSERT statements that populate the tables with initial dummy data for testing: one admin account, three faculty members, and two students.

5.4 requirements.txt - Python Dependencies

A standard pip requirements file listing the two external Python packages needed by the backend:

- supabase: The official Supabase Python client library that provides methods to interact with the Supabase REST API (select, insert, update, delete operations).
- python-dotenv: A utility that reads key-value pairs from a .env file and sets them as environment variables, making configuration management clean and secure.

Install command: `pip install -r requirements.txt`

5.5 .env - Environment Variables

This file stores sensitive configuration that should never be committed to version control. It is listed in .gitignore to prevent accidental exposure.

Required variables:

- SUPABASE_URL: The REST API endpoint URL for your Supabase project. Format: `https://(project-id).supabase.co`
- SUPABASE_KEY: The anonymous (anon) public API key from your Supabase project settings. This key enables read/write access via Row Level Security policies.

The `python-dotenv` library automatically loads these values when `database.py` is imported. If these values are empty or missing, the system gracefully falls back to the mock database.

5.6 backend/README.md

A brief backend-specific readme file providing quick-start instructions for the API server, including environment setup and the command to launch the server.

6. Frontend Directory - Detailed File Analysis

6.1 index.html - Landing Page and Authentication (85 lines)

This is the main entry point of the application. It serves as both the landing page and the authentication gateway. The page operates as a Single Page Application (SPA) where different sections are shown/hidden dynamically using JavaScript without page reloads.

Page sections:

- Institutional Header: Displays the college name ('Zakir Husain College of Engineering and Technology') alongside the college logo image (logo.png).
- Navigation Bar: A dark green bar with direct links to Admin Portal Login, Faculty Portal Login, and Student Portal Login. Clicking these bypasses the role selection screen and goes directly to the login form.
- Banner Strip: A full-width image strip (banner.jpg) displayed below the header. It occupies approximately 45% of the viewport height.
- Hero Section: Contains welcome text and a 'Proceed to Portal' button that reveals the role selection cards.
- Role Selection: Three cards (Administrator, Faculty, Student) that users click to choose their portal.
- Login/Sign Up Section: A dynamic form that adapts based on the selected role. Admin users see only a login form; Faculty and Students see a toggle between Login and Sign Up modes. Sign Up forms capture all database fields required for registration.

The page includes fade-in CSS animations for smooth transitions between sections.

6.2 dashboard.html - Post-Login Dashboard (49 lines)

This page is loaded after a user successfully logs in. It provides the role-specific dashboard interface.

Structure:

- Dashboard Navbar: A green header bar showing the application name and a welcome greeting with the logged-in user's name, plus a Logout button.
- Dashboard Header: Displays the page title (varies by role) along with a search bar (visible for Admin and Faculty) and an 'Add New Record' button (visible only for Admin).
- Content Area: A dynamic div (#content-area) populated entirely by JavaScript based on the user's role.
- Modal Overlay: A hidden modal component used for creating and editing records. Contains a dynamic form (#modal-form) with fields populated by JavaScript at runtime.

6.3 style.css - Complete Stylesheet (432 lines)

The entire visual presentation of the application is defined in this single CSS file. It uses CSS Custom Properties (CSS Variables) defined in the :root selector for consistent theming across all components.

Design System - CSS Variables:

- --bg-color: #f4f5f7 (Light grey page background)
- --accent: #005500 (Primary institutional dark green)
- --accent-hover: #003300 (Darker green for hover states)
- --danger: #cc0000 (Red for delete actions and errors)
- --panel-bg: #ffffff (White background for cards and panels)
- --border-color: #cccccc (Consistent border color)
- --font-family: Arial, Helvetica, sans-serif (System fonts for performance)

Key style categories:

- Layout: Flexbox-based layouts for header, navbar, containers, and dashboard.
 - Components: Styled role cards, authentication forms, profile cards, data tables, modals, buttons (primary/secondary/text), search bars.
- Animation: fadeIn keyframe animation (0.4s ease-out) with translateY for smooth section transitions.
- Banner: Full-viewport-width image strip with 45vh height and object-fit: cover.
- Institutional Theme: Minimal border-radius (2px), flat design, green accent borders, professional typography - all designed to convey a government/institutional aesthetic.

6.4 app.js - Application Logic (477 lines)

This is the most complex file in the project. It contains ALL client-side application logic written in pure Vanilla JavaScript (ES6+). The file handles page routing, authentication, data fetching, table rendering, modal management, and CRUD operations.

Dynamic API URL:

```
const API_URL = "http://" + window.location.hostname + ":8000/api";
```

This line is critical for LAN access. Instead of hardcoding 'localhost', it dynamically reads the current hostname. When accessed from a mobile phone via the laptop's IP address (e.g., 172.25.255.68:8002), the API calls automatically target the correct backend.

Core Functions:

- `initIndex()`: Handles the landing page (`index.html`) logic including section navigation, role selection, login/signup form generation, and form submission.
- `initDashboard()`: Handles the dashboard page - detects user role from `localStorage`, sets up role-specific views, and loads data from the API.
- `setAuthMode(mode)`: Dynamically generates login or signup form fields based on the selected role. Faculty signup includes fields for name, department, designation, email, mobile, specialization, and password. Student signup includes name, course, semester, email, mobile, and password.
- `loadAndRenderTables()`: Fetches faculty and student data from the API, renders HTML tables with search/filter functionality. Admin users see Edit and Delete action buttons.
- `renderStudentProfile()`: Displays a read-only profile card for logged-in students.
- `renderFacultyProfile()`: Displays the faculty member's profile with an Edit button restricted to contact information fields only.
- `openCreateModal()`: Opens the modal with a table-type selector (Faculty/Student) and dynamic form fields for creating new records. Admin-only functionality.
- `openEditModal()`: Opens the modal pre-populated with existing record data for editing. Supports both admin editing (all fields) and self-editing (limited fields).
- `deleteRecord()`: Sends a DELETE request after confirmation prompt. Reloads page on success.
- `activateRoleFlow()`: Shared function used by both navbar links and role cards to initiate the login flow for a specific role.

Session Management:

- Uses `localStorage` to persist user session data (user object and role string).
- On every page load, checks if a session exists. If found on `index.html`, auto-redirects to `dashboard.html`. If missing on `dashboard.html`, redirects back to `index.html`.
- Logout clears `localStorage` and redirects to the landing page.

6.5 logo.png - College Logo Asset

The official college logo image displayed in the top-right corner of the landing page header. If this file is missing or fails to load, a fallback image from Wikipedia (AMU logo) is displayed automatically via the `onerror` handler in the HTML.

Placement: `frontend/logo.png`

6.6 banner.jpg - Banner Image Asset

A full-width horizontal banner image displayed beneath the navigation bar on the landing page. This image strip spans the complete viewport width and occupies approximately 45% of the viewport height. It serves as a visual identity element for the college portal. If this file is missing, the banner section gracefully hides itself via the onerror handler.

Placement: frontend/banner.jpg

6.7 .gitignore (Frontend)

A minimal frontend-specific gitignore file that excludes the .vercel directory (generated during Vercel deployment) from version control.

7. Supabase Integration and Usage

Supabase is used as the cloud-hosted PostgreSQL database for this application. It provides a fully managed database with auto-generated REST APIs.

How the integration works:

- Step 1: Create a project on supabase.com. This provides a unique project URL and API keys.
- Step 2: Run the schema.sql file in the Supabase SQL Editor to create the admin, faculty, and student tables with their respective columns and constraints.
- Step 3: Copy the project URL and anon key into backend/.env.
- Step 4: The database.py file reads these environment variables on startup. If valid, it initializes the Supabase Python client using `create_client(url, key)`.
- Step 5: All CRUD operations (`login`, `get_all`, `create_record`, `update_record`, `delete_record`) are routed through the Supabase client's chained query methods (e.g., `supabase.table('faculty').select('*').eq('email_id', email).execute()`).

To update or replace dummy data in the live database, use the Supabase SQL Editor to run DELETE and INSERT statements directly. Changes reflect immediately in the application without restarting the backend server.

8. Authentication Flow

The authentication process follows these steps:

- 1. User selects a role (Admin/Faculty/Student) via cards or navbar links.
- 2. A dynamic login form is displayed. For Admin: username + password. For Faculty/Student: email + password.
- 3. On form submission, a POST request is sent to `/api/login` with the role, identifier, and password.
- 4. The backend queries the appropriate Supabase table, matching against the identifier and `password_hash` fields.
- 5. On success: the user object (minus `password_hash`) is returned and stored in `localStorage`. The browser redirects to `dashboard.html`.
- 6. On failure: an error message is displayed below the form.
- 7. For Sign Up: a POST request is sent to `/api/{role}` with all form fields. On success, the user is prompted to log in with their new credentials.

9. Role-Based Access Control

Each role has distinct privileges and dashboard views:

Administrator

- Can view ALL faculty and student records in tabular format.
- Can search/filter records using the search bar.
- Can create new Faculty or Student records via the 'Add New Record' modal.
- Can edit any record's fields via the Edit button.
- Can delete any record via the Delete button (with confirmation).
- Cannot sign up - admin accounts must be created directly in the database.

Faculty

- Can view their own profile card with personal details.
- Can edit their own contact information (mobile number, email).
- Can view the Faculty and Student directory tables (read-only).
- Can search/filter records.
- Can self-register via the Sign Up form.

Student

- Can view their own profile card showing name, course, semester, email, mobile.
- Read-only access - cannot modify any data from the dashboard.
- Can self-register via the Sign Up form.

10. Local Network Access (LAN Server)

The application supports access from any device on the same Wi-Fi network, turning the developer's laptop into a local server.

How it works:

- The Python frontend server (`python -m http.server 8002`) binds to 0.0.0.0 by default, accepting connections from any network interface.
- The Python backend server (`server.py`) also binds to all interfaces via `server_address = ('', port)`.
- The JavaScript API URL uses `window.location.hostname` instead of 'localhost', so when a mobile phone accesses the frontend via the laptop's IP (e.g., `http://172.25.255.68:8002`), API requests are automatically directed to `http://172.25.255.68:8000`.
- Both the phone (client) and laptop (server) must be connected to the same Wi-Fi network.

11. How to Run the Project

Prerequisites

- Python 3.x installed on your system.
- A Supabase account with a project created (optional - works with mock data without it).

Step 1: Clone the Repository

```
git clone https://github.com/AzhaanGlitch/ZHCET_Management.git
cd ZHCET_Management
```

Step 2: Set Up Python Virtual Environment

```
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate    # Windows
```

Step 3: Install Backend Dependencies

```
cd backend
pip install -r requirements.txt
```

Step 4: Configure Environment Variables

Create or edit backend/.env with your Supabase credentials:

```
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_KEY=your_anon_key_here
```

Step 5: Initialize the Database

Open your Supabase project dashboard, navigate to the SQL Editor, and run the contents of backend/schema.sql to create the tables and seed data.

Step 6: Start the Backend Server

```
cd backend
python server.py
```

The API server will start on <http://localhost:8000>

Step 7: Start the Frontend Server

```
cd frontend
python -m http.server 8002
```

Open <http://localhost:8002> in your browser to access the application.

Step 8: Access from Mobile (Optional)

Find your laptop's local IP address using 'ip addr' (Linux) or 'ipconfig' (Windows). On your mobile phone connected to the same Wi-Fi, open a browser and navigate to [http://\(laptop-ip\):8002](http://(laptop-ip):8002).